

Comparative Evaluation of NakedBytes, FlatBuffers, and Protocol Buffers for Efficient Data Serialization, Deserialization and Storage

Busoye Tolulope Matthew

December 23, 2025

Abstract

1 Introduction

2 Related Work

3 Methodology

This paper presents a comparative analysis on the three serialization frameworks with the aim to give a holistic analysis of them. The analysis focuses on the following major parameters:

- Resulting Binary: This deals with the resulting storage requirement of the serialized binary output.
- Size: This includes program size, and static ram usage.
- Computation time: This includes the time required for serialization and Deserialization of the data into usage format in the native programming languages.
- Memory Usage: This includes the stack and heap allocation requirement during serialization and Deserialization.
- Compatibility: This includes forward and backward compatibility.

To experiment and analysis were curated to be able to determine the above parameters analytically while removing noise.

The experiment was performed on the following data structure:

The experiment carried out are then divided:

- A simple code that contains utility code and functions and test are carried out to determine the default overhead added.
- A Deserialization alone.
- A Deserialization with one field access.

- A deserialization with all fields access.
- A deserialization with nested fields.
- A serialization alone.
- A serialization with one field access.
- A serialization with all fields access.
- A serialization with nested fields.
- A serialization and Deserialization alone.
- A serialization and Deserialization with one field access.
- A serialization and deserialization with all fields access.
- A serialization and deserialization with nested fields.

These experiment are both performed on the Dell Inc. Precision 5530 Intel® Core™ i9-8950HK × 12 Ubuntu 24.04.3 LTS 64-bit pc and rp2350 Dual Arm Cortex-M33 (150MHz) 520 KB 32 bit.

4 Methodology

This paper resents a comparative experimental evaluation of three serialization frameworks—NakedBytes, FlatBuffers, and Protocol Buffers—with the objective of providing a holistic and practically relevant analysis of their behavior across constrained and general-purpose systems. The evaluation is designed to capture not only serialized size, but also runtime, memory usage, and compatibility characteristics that are critical in energy-sector and embedded deployments.

4.1 Evaluation Parameters

The analysis focuses on the following primary parameters:

- **Resulting Binary Size:** The size of the serialized output buffer produced by each framework for equivalent data structures. This metric captures the efficiency of the on-wire or stored representation, including all internal metadata.
- **Code Size and Static Memory Usage:** This includes the compiled program size (flash/ROM footprint) and static RAM usage introduced by the serialization framework and generated code.
- **Computation Time:** The execution time required for serialization and deserialization operations, measured separately. This includes the cost of parsing, offset resolution, and field access in the native programming language.
- **Memory Usage:** The runtime memory requirements during serialization and deserialization, including:
 - Stack usage

- Heap usage (if any)
- Temporary buffers or scratch memory
- **Compatibility Properties:** The support for forward and backward compatibility, including schema evolution behavior, optional fields, and handling of missing or unknown fields.

These parameters were selected based on prior empirical studies of serialization frameworks and reflect constraints commonly encountered in embedded energy systems, industrial devices, and microcontroller-based platforms.

4.2 Experimental Design

The experiments were carefully curated to isolate individual costs and minimize measurement noise. All benchmarks were conducted using equivalent logical data models and access patterns across frameworks. Where applicable, generated code was used rather than hand-written adapters, reflecting real-world usage.

To enable fine-grained analysis, the experiments were divided into the following categories:

- **Baseline Overhead:** A minimal program containing only framework utilities and generated code, used to determine default code size and static memory overhead.
- **Deserialization Only:**
 - Deserialization without field access
 - Deserialization with single-field access
 - Deserialization with full-field access
 - Deserialization with nested-field access
- **Serialization Only:**
 - Serialization without field access
 - Serialization with single-field access
 - Serialization with full-field access
 - Serialization with nested-field access
- **Combined Serialization and Deserialization:**
 - Serialization followed by deserialization without field access
 - Serialization and deserialization with single-field access
 - Serialization and deserialization with full-field access
 - Serialization and deserialization with nested-field access

This decomposition allows the separation of pure encoding/decoding costs from access-related overhead and highlights the impact of nested structures and variable-length fields.

4.3 Test Data Structures

All experiments were conducted using representative data structures derived from real-world power meter models. These structures include a mix of fixed-length primitive fields, variable-length fields. The sample data structure

Listing 1: Test Data structure

```
struct MeterData
{
    const char *id;
    float voltage;
    float current;
    float power;
    float energy;
    float freq;
    float pf;
    uint16_t alarms;
    const char *timestamp;
};
```

4.4 Experimental Platforms

To evaluate behavior across both high-performance and constrained environments, all experiments were executed on the following platforms:

- **Desktop System:** Dell Inc. Precision 5530, Intel® Core™ i9-8950HK (12 threads), Ubuntu 24.04.3 LTS (64-bit)
- **Embedded System:** RP2350 microcontroller, dual Arm Cortex-M33 at 150 MHz, 520 KB RAM, 32-bit architecture

On both platforms, identical logical benchmarks were executed. Compiler optimizations were fixed across experiments, and measurements were repeated multiple times to ensure consistency. Timing measurements exclude I/O and logging overhead and focus solely on serialization-related computation.

4.5 Experimental Build

The experiment was ran with two different build configuration:

- **Optimization:** Which is -O3 optimizations to enable the highest level of optimization.
- **No Optimization:** Which is -O0 to disable all optimization.

4.6 Measurement Considerations

To reduce noise and ensure reproducibility:

- Warm-up iterations were executed prior to timing measurements.

- Heap allocations were tracked explicitly to detect dynamic memory usage.
- Stack usage was measured using static analysis and runtime watermarking on embedded targets.
- All serialized buffers were validated for correctness after each experiment.

This methodology enables a fair, repeatable, and implementation-independent comparison of serialization frameworks under conditions representative of real-world energy and embedded systems.

5 Methodology

This paper presents a comparative experimental evaluation of three serialization frameworks—NakedBytes, FlatBuffers, and Protocol Buffers—with the objective of providing a holistic and practically relevant analysis of their performance across both constrained embedded systems and general-purpose platforms. The evaluation is designed to capture not only serialized size, but also runtime performance, memory usage, and compatibility characteristics that are critical in energy-sector and industrial deployments.

5.1 Evaluation Parameters

The analysis focuses on the following primary parameters:

- **Resulting Binary Size:** The size of the serialized output buffer produced for equivalent data structures. This metric reflects storage and transmission efficiency, including all framework-specific metadata.
- **Code Size and Static Memory Usage:** The compiled program size (flash/ROM footprint) and static RAM usage introduced by the serialization framework and its generated code.
- **Computation Time:** The execution time required for serialization and deserialization operations. This includes the cost of parsing, offset resolution, and field access in the native programming language.
- **Runtime Memory Usage:** The stack and heap requirements during serialization and deserialization, including temporary buffers and scratch memory.
- **Compatibility Properties:** Support for forward and backward compatibility, including schema evolution, optional fields, and the handling of missing or unknown fields.

5.2 Experimental Design

The experiments were carefully curated to isolate individual costs while minimizing measurement noise. All benchmarks were conducted using equivalent logical data models and access patterns across frameworks. Where applicable, framework-generated code was used instead of hand-written adapters, reflecting real-world usage.

To enable fine-grained analysis, the experiments were divided into the following categories:

- **Baseline Overhead:** A minimal program containing only framework utilities and generated code, used to measure default code size and static memory overhead.
- **Deserialization Only:**
 - Deserialization without field access
 - Deserialization with single-field access
 - Deserialization with full-field access
 - Deserialization with nested-field access
- **Serialization Only:**
 - Serialization without field access
 - Serialization with single-field access
 - Serialization with full-field access
 - Serialization with nested-field access
- **Combined Serialization and Deserialization:**
 - Serialization followed by deserialization without field access
 - Serialization and deserialization with single-field access
 - Serialization and deserialization with full-field access
 - Serialization and deserialization with nested-field access

This decomposition allows separation of pure encoding and decoding costs from field-access overhead, while highlighting the impact of nested structures and variable-length fields.

5.3 Test Data Structure

All experiments were conducted using a representative data structure derived from real-world power meter models. The structure includes a mix of fixed-length primitive fields and variable-length string fields.

Listing 2: Test data structure used for benchmarking

```
struct MeterData
{
    const char *id;
    float voltage;
    float current;
    float power;
    float energy;
    float freq;
    float pf;
    uint16_t alarms;
    const char *timestamp;
};
```

5.4 Experimental Platforms

To evaluate behavior across both high-performance and constrained environments, all experiments were executed on the following platforms:

- **Desktop Platform:** Dell Inc. Precision 5530, Intel® Core™ i9-8950HK (12 threads), Ubuntu 24.04.3 LTS (64-bit)
- **Embedded Platform:** RP2350 microcontroller, dual Arm Cortex-M33 at 150 MHz, 520 KB RAM, 32-bit architecture

Identical logical benchmarks were executed on both platforms. Compiler settings were held constant across experiments, and all measurements were repeated multiple times to ensure consistency.

5.5 Build Configurations

All experiments were performed using two build configurations:

- **Release Build:** Compilation with `-O3` optimization enabled.
- **Debug Build:** Compilation with `-O0` optimization disabled.

5.6 Measurement Considerations

To improve accuracy and reproducibility:

- Warm-up iterations were executed prior to timing measurements.
- Heap allocations were explicitly tracked to detect dynamic memory usage.
- Stack usage was measured using static analysis and runtime watermarking on embedded targets.
- All serialized buffers were validated for correctness after each experiment.

This methodology enables a fair, repeatable, and implementation-independent comparison of serialization frameworks under conditions representative of real-world embedded and energy-system deployments.

6 Results and Discussion

This section presents and analyzes the experimental results obtained from the comparative evaluation of NakedBytes, FlatBuffers, Protocol Buffers, and NanoPB. The discussion focuses on execution time, binary size, and memory footprint, with particular emphasis on how access patterns and build configurations influence overall performance.

Table 1: Deserialization without access with release build

id	avg_time_taken_net(μ s)	min_time_net(μ s)	max_time_net(μ s)	buffer_size_net(B)	flash_text_net(B)	static_ram_net(B)
nakedbytes_deserialization_without_access	-6.5	-114.8	159.2	72	0	0
flatbuffers_deserialization_without_access	-69.4	-141.8	67.2	108	0	0
protobuf_deserialization_without_access	18442.6	14962.2	24099.2	66	2735627	153684
nanopb_deserialization_without_access	54398.6	46610.2	70178.2	66	15609	64

Table 2: Deserialization with one primitive access with release build

id	avg_time_taken_net(μ s)	min_time_net(μ s)	max_time_net(μ s)	buffer_size_net(B)	flash_text_net(B)	static_ram_net(B)
nakedbytes_deserialization_with_one_access	59.00	-105.80	567.20	72.00	42	0
flatbuffers_deserialization_with_one_access	84.50	-36.80	264.20	108.00	189	0
protobuf_deserialization_with_one_access	27517.10	20925.20	36720.20	66.00	2735895	153684
nanopb_deserialization_with_one_access	68305.00	57549.20	78385.20	66.00	15769	64

6.1 Deserialization Performance

Tables 1–3 summarize deserialization performance under release builds for three access patterns: no field access, single-field access, and full-field access.

In the *deserialization without field access* case, NakedBytes and FlatBuffers exhibit near-zero net execution time, reflecting their ability to treat serialized data as an in-place memory view. In contrast, Protocol Buffers and NanoPB incur significantly higher costs due to full message parsing and object reconstruction. Protocol Buffers shows average deserialization times exceeding 18 ms, while NanoPB exceeds 54 ms, even without accessing any fields.

When a *single primitive field* is accessed, NakedBytes and FlatBuffers incur only marginal additional cost, as shown in Table 2. NakedBytes consistently achieves the lowest latency, followed closely by FlatBuffers. Protocol Buffers and NanoPB demonstrate a substantial increase in execution time, indicating that field access triggers additional decoding and data copying operations.

For *full-field access* (Table 3), the performance gap widens further. NakedBytes maintains sub-millisecond execution times, while FlatBuffers exhibits a moderate increase due to offset resolution and accessor logic. Protocol Buffers and NanoPB remain one to two orders of magnitude slower, reflecting the cumulative cost of decoding all fields into runtime representations.

Overall, these results demonstrate that zero-copy and offset-based designs scale more favor-

Table 3: Deserialization with all field access with release build

id	avg_time_taken_net(μ s)	min_time_net(μ s)	max_time_net(μ s)	buffer_size_net(B)	flash_text_net(B)	static_ram_net(B)
nakedbytes_deserialization_with_all_access	348.50	143.20	538.20	72.00	392	0
flatbuffers_deserialization_with_all_access	725.00	494.20	1003.20	108.00	768	0
protobuf_deserialization_with_all_access	35842.00	22141.20	51442.20	66.00	2736059	153684
nanopb_deserialization_with_all_access	64629.80	59865.20	72463.20	66.00	15961	64

Table 4: Serialization with one primitive field with release build

id	avg_time_taken_net(μ s)	min_time_net(μ s)	max_time_net(μ s)	buffer_size_net(B)	flash_text_net(B)	static_ram_net(B)
nakedbytes_serialization_primitives	-32.50	-63.20	161.80	36.00	121	0
flatbuffers_serialization_primitives	9632.20	7874.80	14659.80	56.00	21489	96
protobuf_serialization_primitives	7586.00	6645.80	8430.80	33.00	2736727	153724
nanopb_serialization_primitives	39401.30	35642.80	46088.80	33.00	26332	80

ably with access complexity compared to parse-and-materialize approaches.

6.2 Serialization Performance

Serialization results for release builds are presented in Tables 4 and 5. NakedBytes consistently achieves the lowest serialization time, particularly when serializing primitive fields or the full data structure. This behavior is attributed to its direct in-place buffer construction, which avoids intermediate representations and dynamic memory allocation.

FlatBuffers shows significantly higher serialization costs, especially for full-structure serialization. This overhead arises from backward buffer construction, object graph traversal, and builder management. Although FlatBuffers provides efficient deserialization, its serialization path is comparatively expensive.

Protocol Buffers exhibits moderate serialization times but incurs substantial code size and static memory overhead, as discussed in Section ???. NanoPB shows the highest serialization latency among embedded-focused frameworks, primarily due to field-by-field encoding and explicit bounds checking.

6.3 Impact of Debug Builds

Tables 6–10 present corresponding results under debug (`-O0`) builds. Across all frameworks, execution time increases significantly due to the absence of compiler optimizations. However, the relative performance ordering remains consistent.

Table 5: Serialization with all field access with release build

id	avg_time_taken_net(μ s)	min_time_net(μ s)	max_time_net(μ s)	buffer_size_net(B)	flash_text_net(B)	static_ram_net(B)
nakedbytes_serialization_all	-40.60	-68.20	56.80	71.00	121	0
flatbuffers_serialization_all	11628.70	9181.80	19015.80	108.00	21841	96
protobuf_serialization_all	7738.10	7071.80	9851.80	66.00	2736855	153724
nanopb_serialization_all	52952.50	46314.80	72372.80	66.00	26412	80

Table 6: Deserialization without access with debug build

id	avg_time_taken_net(μ s)	min_time_net(μ s)	max_time_net(μ s)	buffer_size_net(B)	flash_text_net(B)	static_ram_net(B)
nakedbytes_deserialization_without_access	-3.40	-58.10	393.90	72.00	13	0
flatbuffers_deserialization_without_access	1709.70	1423.90	2682.90	108.00	747	16
protobuf_deserialization_without_access	51975.60	45389.90	61155.90	66.00	2760517	153876
nanopb_deserialization_without_access	146932.50	130173.90	173290.90	66.00	16656	64

NakedBytes continues to outperform other frameworks even in debug builds, demonstrating that its performance advantages are largely structural rather than compiler-dependent. FlatBuffers experiences a pronounced slowdown, particularly during full-field deserialization, indicating sensitivity to optimization levels. Protocol Buffers and NanoPB suffer the largest absolute slowdowns, further emphasizing the cost of parsing-heavy designs in unoptimized builds.

These results are particularly relevant for embedded development workflows, where debug builds are frequently used during testing and validation.

6.4 Binary Size and Memory Footprint

Across all experiments, NakedBytes and FlatBuffers maintain small and predictable serialized buffer sizes. NakedBytes produces slightly larger buffers than Protocol Buffers in some cases due to explicit offset storage, but this overhead remains constant and independent of access patterns.

Protocol Buffers achieves compact wire representations; however, this advantage is offset by its substantial flash and static RAM usage. On embedded targets, Protocol Buffers consistently requires over 2.7 MB of flash and more than 150 KB of static RAM, rendering it impractical for microcontroller-class devices. NanoPB significantly reduces this footprint but still incurs higher runtime costs compared to NakedBytes and FlatBuffers.

Notably, NakedBytes introduces negligible static RAM usage across all experiments, reinforcing its suitability for memory-constrained environments.

Table 7: Deserialization with one primitive access with debug build

id	avg_time_taken_net(μ s)	min_time_net(μ s)	max_time_net(μ s)	buffer_size_net(B)	flash_text_net(B)	static_ram_net(B)
nakedbytes_deserialization_with_one_access	415.90	335.90	821.90	72.00	714	8
flatbuffers_deserialization_with_one_access	6050.90	5251.90	6923.90	108.00	1974	16
protobuf_deserialization_with_one_access	53187.20	45772.90	63279.90	66.00	2761313	153884
nanopb_deserialization_with_one_access	147070.20	130518.90	178206.90	66.00	17302	72

Table 8: Deserialization with all field access with debug build

id	avg_time_taken_net(μ s)	min_time_net(μ s)	max_time_net(μ s)	buffer_size_net(B)	flash_text_net(B)	static_ram_net(B)
nakedbytes_deserialization_with_all_access	2950.90	2486.90	3617.90	72.00	2922	16
flatbuffers_deserialization_with_all_access	40084.40	36615.90	49652.90	108.00	4473	24
protobuf_deserialization_with_all_access	62634.90	48593.90	78255.90	66.00	2763346	153892
nanopb_deserialization_with_all_access	142805.60	127838.90	169998.90	66.00	18621	72

6.5 Discussion Summary

The experimental results highlight clear trade-offs among the evaluated serialization frameworks:

- NakedBytes consistently achieves the lowest serialization and deserialization latency, minimal static memory usage, and predictable binary layouts.
- FlatBuffers provides efficient zero-copy deserialization but incurs higher serialization costs and moderate code-size overhead.
- Protocol Buffers prioritizes portability and schema evolution at the expense of execution time and memory footprint.
- NanoPB reduces Protocol Buffers’ memory footprint but remains significantly slower than offset-based approaches.

These findings suggest that offset-oriented, in-place serialization frameworks are better suited for embedded and energy-sector applications where deterministic performance, low latency, and minimal memory usage are critical.

Table 9: Serialization with all field access with debug build

id	avg_time_taken_net(μ s)	min_time_net(μ s)	max_time_net(μ s)	buffer_size_net(B)	flash_text_net(B)	static_ram_net(B)
nakedbytes_serialization_primitives	9759.10	8843.90	12540.90	36.00	4006	32
flatbuffers_serialization_primitives	133820.70	126069.90	146085.90	56.00	21675	256
protobuf_serialization_primitives	80508.00	73279.90	94318.90	33.00	2765006	153916
nanopb_serialization_primitives	88060.00	82532.90	95782.90	33.00	27044	72

Table 10: Serialization with all field access with release build

id	avg_time_taken_net(μ s)	min_time_net(μ s)	max_time_net(μ s)	buffer_size_net(B)	flash_text_net(B)	static_ram_net(B)
nakedbytes_serialization_all	16715.00	15042.90	18930.90	71.00	5284	56
flatbuffers_serialization_all	206981.50	191487.90	242319.90	108.00	23261	264
protobuf_serialization_all	79288.00	73385.90	98133.90	66.00	2766331	153924
nanopb_serialization_all	122513.40	105399.90	140454.90	66.00	27446	72

7 Conclusion